CS 300 Project One

Pseudocode and Runtime Analysis

Gabriel Collins

Southern New Hampshire University

Course Object and File Validation

```
DEFINE Course
    courseNumber: string
    courseTitle: string
    prerequisites: vector of strings
END Course

FUNCTION ReadCourseFile(fileName)
    OPEN fileName
    IF file cannot be opened THEN DISPLAY error; RETURN failure
    CREATE empty fileLines and validCourseNumbers
    WHILE another line exists
        READ line; IF blank THEN CONTINUE
        SPLIT line by comma into tokens; TRIM each token
        IF fewer than 2 tokens OR number/title blank THEN
            DISPLAY "Invalid file format"; CLOSE file; RETURN failure
        END IF
        IF number already in validCourseNumbers THEN
            DISPLAY "Duplicate course"; CLOSE file; RETURN failure
        END IF
        ADD tokens to fileLines
        ADD course number to validCourseNumbers
    END WHILE
    CLOSE file
    FOR EACH tokens IN fileLines
        FOR EACH prerequisite token after title
            IF blank OR not in validCourseNumbers THEN
                DISPLAY "Invalid prerequisite"; RETURN failure
            END IF
        END FOR
    END FOR
    RETURN fileLines
END FUNCTION

FUNCTION CreateCourse(tokens)
    CREATE new Course called course
    SET course.courseNumber = tokens[0]
    SET course.courseTitle = tokens[1]
    FOR EACH token after tokens[1]
        ADD token to course.prerequisites
    END FOR
    RETURN course
END FUNCTION
```

Vector Data Structure

```
FUNCTION LoadVector(fileName, courses)
    SET lines = ReadCourseFile(fileName)
    IF lines is failure THEN RETURN false
    CLEAR courses
    FOR EACH tokens IN lines
        ADD CreateCourse(tokens) to courses
    END FOR
    RETURN true
END FUNCTION

FUNCTION PrintVectorList(courses)
    COPY courses to sortedCourses
    SORT sortedCourses by courseNumber ascending
    FOR EACH course IN sortedCourses
        DISPLAY course.courseNumber + ", " + course.courseTitle
    END FOR
END FUNCTION

FUNCTION FindVector(courses, number)
    FOR EACH course IN courses
        IF course.courseNumber equals number THEN RETURN course
    END FOR
    RETURN null
END FUNCTION
```

Hash Table Data Structure

```
FUNCTION LoadHash(fileName, courseTable)
    SET lines = ReadCourseFile(fileName)
    IF lines is failure THEN RETURN false
    CLEAR courseTable
    FOR EACH tokens IN lines
        SET course = CreateCourse(tokens)
        INSERT course into courseTable using course.courseNumber
    END FOR
    RETURN true
END FUNCTION

FUNCTION PrintHashList(courseTable)
    CREATE empty sortedCourses
    FOR EACH bucket IN courseTable
        FOR EACH course IN bucket
            ADD course to sortedCourses
        END FOR
    END FOR
    SORT sortedCourses by courseNumber ascending
    FOR EACH course IN sortedCourses
        DISPLAY course.courseNumber + ", " + course.courseTitle
    END FOR
END FUNCTION
```

```
FUNCTION FindHash(courseTable, number)
    RETURN SEARCH courseTable using number
END FUNCTION
```

Binary Search Tree Data Structure

```
DEFINE Node: course as Course, left as Node, right as Node
DEFINE BinarySearchTree: root as Node

FUNCTION LoadTree(fileName, courseTree)
    SET lines = ReadCourseFile(fileName)
    IF lines is failure THEN RETURN false
    SET courseTree.root = null
    FOR EACH tokens IN lines
        CALL InsertTree(courseTree, CreateCourse(tokens))
    END FOR
    RETURN true
END FUNCTION

FUNCTION InsertTree(courseTree, course)
    CREATE newNode containing course, left=null, right=null
    IF courseTree.root is null THEN
        SET courseTree.root = newNode; RETURN
    END IF
    SET current = courseTree.root
    WHILE true
        IF course.courseNumber < current.course.courseNumber THEN
            IF current.left is null THEN
                SET current.left = newNode; RETURN
            END IF
            SET current = current.left
        ELSE
            IF current.right is null THEN
                SET current.right = newNode; RETURN
            END IF
            SET current = current.right
        END IF
    END WHILE
END FUNCTION

FUNCTION PrintTreeList(node)
    IF node is not null THEN
        CALL PrintTreeList(node.left)
        DISPLAY node.course.courseNumber + ", " + node.course.courseTitle
        CALL PrintTreeList(node.right)
    END IF
END FUNCTION

FUNCTION FindTree(courseTree, number)
    SET current = courseTree.root
    WHILE current is not null
        IF number equals current.course.courseNumber THEN
            RETURN current.course
```

```
        ELSE IF number < current.course.courseNumber THEN
            SET current = current.left
        ELSE
            SET current = current.right
        END IF
    END WHILE
    RETURN null
END FUNCTION
```

Individual Course and Prerequisite Information

```
FUNCTION PrintCourseInformation(structure, number, findFunction)
    SET course = CALL findFunction(structure, number)
    IF course is null THEN DISPLAY "Course not found"; RETURN
    DISPLAY course.courseNumber + ", " + course.courseTitle
    IF course.prerequisites is empty THEN
        DISPLAY "Prerequisites: None"
    ELSE
        DISPLAY "Prerequisites:"
        FOR EACH prerequisiteNumber IN course.prerequisites
            SET prerequisite = CALL findFunction(structure, prerequisiteNumber)
            IF prerequisite is not null THEN
                DISPLAY prerequisite.courseNumber + ", " + prerequisite.courseTitle
            ELSE
                DISPLAY prerequisiteNumber
            END IF
        END FOR
    END IF
END FUNCTION
```

Main Program and Menu

```
BEGIN Main
    SELECT structureType (Vector, Hash Table, or Binary Search Tree)
    CREATE empty selected structure; SET fileLoaded = false
    REPEAT
        DISPLAY "1. Load Data Structure"
        DISPLAY "2. Print Course List"
        DISPLAY "3. Print Course"
        DISPLAY "9. Exit"
        INPUT choice
        IF choice equals 1 THEN
            INPUT fileName
            IF structureType = Vector THEN CALL LoadVector
            ELSE IF structureType = Hash Table THEN CALL LoadHash
            ELSE CALL LoadTree
            SET fileLoaded to the result; DISPLAY success or failure
        ELSE IF choice equals 2 OR choice equals 3 THEN
            IF NOT fileLoaded THEN DISPLAY "Load data first"
            ELSE IF choice equals 2 THEN
                IF Vector THEN CALL PrintVectorList
                ELSE IF Hash Table THEN CALL PrintHashList
                ELSE CALL PrintTreeList(root)
            ELSE
```

```
                INPUT requestedCourseNumber
                SELECT FindVector, FindHash, or FindTree for structureType
                CALL PrintCourseInformation with selected Find function
            END IF
        ELSE IF choice is not 9 THEN DISPLAY "Invalid option"
        END IF
    UNTIL choice equals 9
END Main
```

Worst-Case Runtime Evaluation

Let n be the number of courses and k the maximum prerequisites per course (treated as constant). Costs of basic statements are 1; search and insert calls use their algorithmic cost. Only file loading and course creation are included, as required.

Statement / operation	Cost each	Executions	Total
OPEN and verify file	1	1	O(1)
CREATE temporary lists	1	1	O(1)
READ each line	1	n	O(n)
SPLIT / TRIM / field validation	1	n	O(n)
Duplicate membership in list	O(n)	n	$O(n^2)$
ADD line and course number	1	n	O(n)
CLOSE file	1	1	O(1)
Prerequisite membership in list	O(n)	at most kn	$O(n^2)$
CREATE Course and assign fields	1	n	O(n)
COPY prerequisites to Course	1	at most kn	O(n)
Vector append (amortized)	O(1)	n	O(n)
Hash insertion (worst-case)	O(n)	n	$O(n^2)$
BST insertion (worst-case)	O(n)	n	$O(n^2)$

The shared list-based validation contributes $O(n^2)$ worst-case time, so total worst-case loading is $O(n^2)$ for each structure. Hash insertion averages O(1) per course; BST insertion averages O(log n) in a reasonably balanced tree. The course records, temporary lines, and validation list use O(n) memory; the hash table has bucket overhead, and BST nodes have pointer overhead.

Data Structure Comparison

Structure	Find course	Sorted list	Space
Vector	O(n)	O(n log n) sorting	O(n)
Hash table	O(1) avg.; O(n) worst	O(n log n) sorting	O(n)
BST	O(log n) avg.; O(n) worst	O(n) in-order	O(n)

The vector is the easiest option for me to follow because courses are stored in a straightforward list. The downside is that I would have to search through that list to find a course and sort it before displaying courses in order.

A hash table makes sense when an advisor wants to look up one course quickly. However, the courses would not automatically be in alphanumeric order, and collisions could make some searches slower.

The binary search tree stands out because an in-order traversal already gives the course list in the order the advisors need. Searching can also be fairly quick. One drawback is that if the tree becomes unbalanced, searching and inserting courses can take much longer.

Recommendation

I would choose the binary search tree for this project. The main reason is that the advisors need to see every course in alphanumeric order, and an in-order traversal handles that without sorting another list first. It also supports looking up a specific course and its prerequisites. I would need to keep an eye on how balanced the tree is, though. If ABCU had a much larger course list, a self-balancing tree would be a better long-term choice.